

Отчёт по задаче построения множества $EFF_k(\alpha)$

Автор: Речкин Вадим Сергеевич

Группа: 23Б08-мм

GitHub: github.com/VadosikRRR/.../EFF.cpp

1. Формулировка задачи

Для КС-грамматики, параметра $k \geq 1$ и цепочки α требуется построить множество $EFF_k(\alpha)$:

1. Рассмотреть терминальные цепочки, получаемые правосторонними выводами из α .
2. Для каждой цепочки взять всю цепочку при длине меньше k , иначе — префикс длины k .
3. Добавить результат в $EFF_k(\alpha)$ только если последнее правило вывода не является ε -продукцией.

2. Принятый формат входа

Из-за отсутствия строго зафиксированного формата в тексте задачи в программе принят следующий:

```
m
LHS1 RHS1
...
LHSm RHSm
alpha
k
maxSteps maxFormLen
```

- m — число правил;
- правило вида LHS RHS, где LHS — один нетерминал $A \in Z$;
- RHS = `eps` означает ε -продукцию;
- α — начальная цепочка;
- k — параметр из задачи;
- maxSteps, maxFormLen — технические ограничения перебора.

3. Идея решения

Используется обход в ширину по пространству правосторонних выводов:

1. состояние: текущая цепочка, число шагов, флаг *было ли последнее правило epsilon*;
2. на каждом шаге раскрывается **правый** нетерминал;
3. применяются все правила для этого нетерминала;
4. если получена терминальная цепочка и последнее правило не epsilon, в ответ добавляется либо вся цепочка (если $|w| < k$), либо префикс длины k .

Для предотвращения бесконечного перебора добавлены ограничения по числу шагов и длине промежуточных цепочек.

4. Код программы

```
#include <bits/stdc++.h>
using namespace std;

struct Rule {
    char lhs;
    string rhs; // empty string means epsilon
};

static bool isNonTerminal(char c) {
    return c >= 'A' && c <= 'Z';
}

static int rightmostNonTerminalPos(const string &s) {
    for (int i = (int)s.size() - 1; i >= 0; --i) {
        if (isNonTerminal(s[i])) return i;
    }
    return -1;
}

static bool isTerminalString(const string &s) {
    for (char c : s) {
        if (isNonTerminal(c)) return false;
    }
    return true;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    ifstream fin("example_input.txt");

    int m;
    fin >> m;
```

```

vector<Rule> rules(m);
for (int i = 0; i < m; ++i) {
    string lhs, rhs;
    fin >> lhs >> rhs;
    rules[i].lhs = lhs[0];
    rules[i].rhs = (rhs == "eps" ? "" : rhs);
}

string alpha;
int k, maxSteps, maxFormLen;
fin >> alpha;
fin >> k;
fin >> maxSteps >> maxFormLen;

vector<vector<Rule>> byLhs(26);
for (const Rule &r : rules) {
    if (isNonTerminal(r.lhs)) {
        byLhs[r.lhs - 'A'].push_back(r);
    }
}

struct State {
    string form;
    int steps;
    int lastRuleWasEps; // 1 if last applied rule was epsilon, 0
                     otherwise, -1 for start
};

queue<State> q;
q.push({alpha, 0, -1});

// We keep visited by (form, steps) to avoid repeated exploration
loops on the same layer.
set<pair<string, int>> visited;
visited.insert({alpha, 0});

set<string> eff;

while (!q.empty()) {
    State cur = q.front();
    q.pop();

    if (cur.steps > maxSteps) continue;

    if (isTerminalString(cur.form)) {
        if (cur.steps > 0 && cur.lastRuleWasEps == 0) {
            string out = cur.form;
            if ((int)out.size() > k) out = out.substr(0, k);
            eff.insert(out);
        }
        continue;
    }
}

```

```

        if (cur.steps == maxSteps) continue;

        int pos = rightmostNonTerminalPos(cur.form);
        if (pos == -1) continue;

        char nt = cur.form[pos];
        const vector<Rule> &cand = byLhs[nt - 'A'];

        for (const Rule &r : cand) {
            string next = cur.form.substr(0, pos) + r.rhs + cur.form.
                substr(pos + 1);
            if ((int)next.size() > maxFormLen) continue;

            int nextSteps = cur.steps + 1;
            pair<string, int> key = {next, nextSteps};
            if (visited.insert(key).second) {
                q.push({next, nextSteps, r.rhs.empty() ? 1 : 0});
            }
        }
    }

    cout << "EFF_" << k << "(" << alpha << "):" << '\n';
    if (eff.empty()) {
        cout << "<empty>" << '\n';
    } else {
        for (const string &s : eff) {
            cout << s << '\n';
        }
    }

    return 0;
}

```

5. Оценка сложности

Пусть:

- R — число правил;
- D — `maxSteps`;
- L — `maxFormLen`;
- S — число достижимых состояний (`form`, `steps`) в заданных ограничениях.

Тогда оценка:

- время: $O(S \cdot R \cdot L)$ в верхней границе;
- память: $O(S \cdot L)$ на хранение посещённых цепочек и очереди.

На практике сложность определяется выбранными ограничениями `maxSteps` и `maxFormLen`.

6. Пример

Вход:

```
8
S aA
S bB
A c
A eps
B dC
B eps
C e
C eps
S
2
12 30
```

Выход:

```
EFF_2(S):
ac
bd
```

Пояснение:

- вывод $S \Rightarrow aA \Rightarrow ac$ даёт **ac**;
- вывод $S \Rightarrow bB \Rightarrow bdC \Rightarrow bde$ даёт префикс **bd**;
- цепочки, у которых последнее правило — epsilon, в ответ не включаются.